

T-Sharp (T#) — Dosya İşlemleri

İçindekiler

1. Dosya Okuma — `dosya_oku()`
2. Dosya Yazma — `dosya_yaz()`
3. Dosyaya Ekleme — `dosya_ekle()`
4. Dosya Varlık Kontrolü — `dosya_var_mi()`
5. Klasör Oluşturma — `klasor_olustur()`
6. ZIP İşlemleri
7. Tam Örnek Programlar

1. Dosya Okuma — `dosya_oku()`

Bir dosyanın tüm içeriğini okuyup bir değişkene atar.

Söz dizimi

```
degisken <degisken_adi> = dosya_oku("<dosya_yolu>")
```

Örnekler

```
// Basit okuma
degisken icerik = dosya_oku("notlar.txt")
yazdır icerik
```

```
// Değişkenden yol ile okuma
degisken dosya_yolu = "veriler/liste.txt"
degisken metin = dosya_oku(dosya_yolu)
yazdır metin
```

```
// Satır satır işleme (bol ile)

degisken icerik = dosya_oku("isimler.txt")

liste satirlar = bol(icerik, "\n")

her satir icinde satirlar:
    eger uzunluk(satir) buyuktur 0:
        yazdır satir
    son
son
```

Not: Dosya UTF-8 kodlamasıyla okunur. Dosya bulunamazsa `hic` döner ve terminale hata mesajı yazdırılır. Program çökmez.

2. Dosya Yazma — `dosya_yaz()`

Bir dosyaya içerik yazar. Dosya **yoksa oluşturur, varsa üzerine yazar** (siler, sıfırdan başlar).

Söz dizimi

```
dosya_yaz("<dosya_yolu>", "<icerik>")
```

Sonucu bir değişkene atarak başarılı olup olmadığını kontrol edebilirsiniz:

```
degisken basarili = dosya_yaz("<dosya_yolu>", "<icerik>")
```

Örnekler

```
// Basit yazma

dosya_yaz("cikti.txt", "Merhaba, T-Sharp!")
```

```
// Değişken içeriği yazma

degisken isim = "Talha"
degisken yas = 20
degisken satir = isim + " - " + yaziya(yas)
dosya_yaz("kisi.txt", satir)
```

```
// Çok satırlı içerik yazma
degisken icerik = "Birinci satır\nİkinci satır\nÜçüncü satır"
dosya_yaz("cok_satirli.txt", icerik)
```

```
// Başarı kontrolü
degisken basarili = dosya_yaz("log.txt", "İşlem tamamlandı.")
eger basarili:
    yazdır "Dosya başarıyla yazıldı."
degilse:
    yazdır "Dosya yazılamadı!"
son
```

```
// Alt klasöre yazma (klasör yoksa otomatik oluşturulur)
dosya_yaz("sonuclar/rapor.txt", "Bu bir rapordur.")
```

3. Dosyaya Ekleme — `dosya_ekle()`

Mevcut bir dosyanın **sonuna** içerik ekler. Dosya yoksa oluşturur.

Söz dizimi

```
dosya_ekle("<dosya_yolu>", "<eklenecek_icerik>")
```

Örnekler

```
// Log dosyasına satır ekleme
dosya_ekle("log.txt", "Kullanıcı giriş yaptı.\n")
dosya_ekle("log.txt", "İşlem başlatıldı.\n")
dosya_ekle("log.txt", "İşlem tamamlandı.\n")
```

```
// Değişken değeri ekleme
degisken hata_mesaji = "Bağlantı zaman aşımına uğradı"
dosya_ekle("hatalar.txt", hata_mesaji + "\n")
```

```
// Döngü ile veri biriktirme
liste isimler ["Ali", "Veli", "Ayşe", "Fatma"]

her isim icinde isimler:
    dosya_ekle("isimler.txt", isim + "\n")
son

yazdır "Tüm isimler dosyaya yazıldı."
```

Fark: `dosya_yaz()` dosyayı sıfırlar. `dosya_ekle()` mevcut içeriği korur, sona ekler.

4. Dosya Varlık Kontrolü — `dosya_var_mi()`

Belirtilen dosyanın veya klasörün var olup olmadığını `dogru` / `yanlis` olarak döner.

Söz dizimi

```
degisken <sonuc> = dosya_var_mi("<dosya_yolu>")
```

Örnekler

```
degisken var_mi = dosya_var_mi("ayarlar.txt")

eger var_mi:
    degisken icerik = dosya_oku("ayarlar.txt")
    yazdır "Ayarlar yüklendi:"
    yazdır icerik
degilse:
    yazdır "Ayar dosyası bulunamadı, varsayılanlar kullanılıyor."
    dosya_yaz("ayarlar.txt", "dil=turkce\ntema=acik")
son
```

```
// Üzerine yazmadan önce kontrol
degisken hedef = "onemli_veri.txt"

eger dosya_var_mi(hedef):
    yazdır "UYARI: Bu dosya zaten var, üzerine yazılacak!"
    girdi onay "Devam etmek istiyor musunuz? (e/h): "
    eger onay esittir "e":
        dosya_yaz(hedef, "Yeni içerik")
        yazdır "Dosya güncellendi."
    son
degilse:
    dosya_yaz(hedef, "Yeni içerik")
    yazdır "Yeni dosya oluşturuldu."
son
```

5. Klasör Oluşturma — `klasor_olustur()`

Belirtilen yolda bir klasör oluşturur. Zaten varsa hata vermez.

Söz dizimi

```
klasor_olustur("<klasor_yolu>")
```

Örnekler

```
// Tek klasör
klasor_olustur("ciktilar")
yazdır "Klasör hazır."
```

```
// İç içe klasör yapısı
klasor_olustur("proje/veriler/ham")
klasor_olustur("proje/veriler/islenmis")
klasor_olustur("proje/raporlar")

// Artık bu klasörlere dosya yazılabilir
dosya_yaz("proje/veriler/ham/giris.txt", "Ham veri burada.")
dosya_yaz("proje/raporlar/rapor_1.txt", "Rapor içeriği.")
```

6. ZIP İşlemleri

T# v4.1, ZIP arşiv dosyalarını oluşturma, açma ve yönetme imkânı sunar.

6.1 ZIP Oluşturma — `zip_olustur`

Boş bir ZIP dosyası oluşturur.

```
zip_olustur "arsiv.zip"
```

6.2 ZIP'e Dosya Ekleme — `zip_ekle`

Var olan bir ZIP'e dosya ekler.

```
zip_ekle "<zip_dosyasi>" "<eklenecek_dosya>" ["arsiv_icerisindeki_ad"]
```

```
// ZIP oluştur ve dosyaları ekle
zip_olustur "yedek.zip"
zip_ekle "yedek.zip" "notlar.txt"
zip_ekle "yedek.zip" "veriler.csv"
zip_ekle "yedek.zip" "resimler/foto.png" "foto.png"

yazdır "Yedekleme tamamlandı!"
```

6.3 ZIP İçeriğini Listeleme — `zip_listele`

ZIP içindeki dosyaları listeler ve bir listeye atar.

```
zip_listele "<zip_dosyasi>" <degisken_adi>
```

```
zip_listele "arsiv.zip" dosya_listesi
yazdır "Arşivdeki dosya sayısı:" uzunluk(dosya_listesi)
```

6.4 ZIP Açma — `zip_ac`

Bir ZIP dosyasını açıp değişkene bağlar (işlem için handle).

```
zip_ac "<zip_dosyasi>" <degisken_adi>
```

```
zip_ac "arsiv.zip" arsip_handle
```

6.5 ZIP'ten Dosya Çıkarma — `zip_cikar`

ZIP içindeki dosyaları bir klasöre çıkarır.

```
zip_cikar "<zip_dosyasi>" "<hedef_klasor>" ["sadece_bu_dosya"]
```

```
// Tüm içeriği çıkar
zip_cikar "arsiv.zip" "cikartilan_dosyalar"

// Tek dosya çıkar
zip_cikar "arsiv.zip" "cikartilan_dosyalar" "notlar.txt"
```

6.6 ZIP Silme — `zip_sil`

ZIP dosyasını diskten siler.

```
zip_sil "eski_arsiv.zip"
yazdır "Eski arşiv silindi."
```

ZIP — Tam Örnek

```
// Birden fazla dosyayı arşivle
liste dosyalar ["rapor.txt", "veri.csv", "ayarlar.txt"]

// Önce dosyaları oluştur (örnek amaçlı)
dosya_yaz "rapor.txt" "Bu bir rapordur."
dosya_yaz "veri.csv" "ad,yas\nAli,25\nVeli,30"
dosya_yaz "ayarlar.txt" "tema=karanlik"

// ZIP oluştur
zip_olustur "paket.zip"

// Dosyaları ekle
her dosya icinde dosyalar:
    eger dosya_var_mi(dosya):
        zip_ekle "paket.zip" dosya
        yazdir dosya "eklendi."
    degilse:
        yazdir dosya "bulunamadı, atlandı."
son

// İçeriği listele
yazdir "--- Arşiv içeriği ---"
zip_listele "paket.zip" liste_sonuc

yazdir "Toplam" uzunluk(liste_sonuc) "dosya arşivlendi."
```


7. Tam Örnek Programlar

Örnek 1 — Basit Not Defteri

```
// Not defteri: kaydet ve oku

degisken dosya = "notlar.txt"

dongu dogru:
    yazdır "=== NOT DEFTERİ ==="
    yazdır "1) Not ekle"
    yazdır "2) Notları görüntüle"
    yazdır "3) Çıkış"
    yazdır
    girdi secim "Seçiminiz: "

    eger secim esittir "1":
        girdi yeni_not "Notunuzu girin: "
        dosya_ekle dosya yeni_not + "\n"
        yazdır "Not kaydedildi!"
    son

    eger secim esittir "2":
        eger dosya_var_mi(dosya):
            degisken icerik = dosya_oku(dosya)
            yazdır "--- Notlarınız ---"
            yazdır icerik
        degilse:
            yazdır "Henüz hiç not yok."
        son
    son

    eger secim esittir "3":
        yazdır "Güle güle!"
        dur
    son

son
```

Örnek 2 — CSV Okuma ve İşleme

```
// Öğrenci CSV dosyasını oku ve ortalamaları hesapla

// Örnek CSV oluştur
dosya_yaz "ogrenciler.csv" "isim,mat,fen,turkce\nAli,80,90,75\nAyşe,95,88,92\nVeli,60,70,65"

// Oku
degisken icerik = dosya_oku("ogrenciler.csv")
liste satirlar = bol(icerik, "\n")

// Başlık satırını atla (ilk eleman)
degisken baslik = satirlar[0]
yazdır "Sütunlar:" baslik
yazdır "---"

degisken i = 1
dongu i kucuktur uzunluk(satirlar):
    degisken satir = satirlar[i]
    eger uzunluk(satir) buyuktur 0:
        liste kolonlar = bol(satir, ",")
        degisken isim = kolonlar[0]
        degisken mat  = ondalikliya(kolonlar[1])
        degisken fen  = ondalikliya(kolonlar[2])
        degisken tur  = ondalikliya(kolonlar[3])
        degisken ort  = yuvarla((mat + fen + tur) / 3, 2)
        yazdır isim "→ Ortalama:" ort
    son
    i += 1
son
```

Örnek 3 — Otomatik Log Sistemi

```
// Tarihli log sistemi

fonksiyon log_yaz(seviye, mesaj):
    degisken zaman_damgasi = yaziya(zaman())
    degisken log_satiri = "[" + seviye + "]" + mesaj + "\n"
    dosya_ekle "uygulama.log" log_satiri
    yazdır log_satiri
son

fonksiyon log_temizle():
    dosya_yaz "uygulama.log" ""
    yazdır "Log temizlendi."
son

// Kullanım
log_temizle()
log_yaz("BILGI", "Uygulama başlatıldı.")
log_yaz("BILGI", "Kullanıcı doğrulandı.")
log_yaz("UYARI", "Disk doluluk oranı %80 üzerinde.")
log_yaz("HATA", "Veritabanına bağlanılamadı.")

yazdır "--- LOG DOSYASI ---"
degisken log_icerik = dosya_oku("uygulama.log")
yazdır log_icerik
```

Örnek 4 — Klasör Yapısı Oluşturucu

```
// Proje iskeletini otomatik kur

degisken proje_adi = "yeni_proje"

liste klasorler [
    proje_adi + "/src",
    proje_adi + "/veriler",
    proje_adi + "/ciktilar",
    proje_adi + "/docs"
]

her klasor icinde klasorler:
    klasor_olustur(klasor)
    yazdır "✓" klasor "oluşturuldu"
son

// Başlangıç dosyaları oluştur
dosya_yaz proje_adi + "/src/ana.tsharp" "// Ana program dosyası\nyazdır \"Proje hazır!\"
dosya_yaz proje_adi + "/docs/README.md" "# \" + proje_adi + \"\n\nBu proje T-Sharp ile yazılmıştır.\"

yazdır
yazdır "Proje yapısı hazır!"
```

Hızlı Başvuru Tablosu

Komut / Fonksiyon	Açıklama
<code>dosya_oku("yol")</code>	Dosyayı okur, metin döner
<code>dosya_yaz("yol", "icerik")</code>	Dosyaya yazar (sıfırdan)
<code>dosya_ekle("yol", "icerik")</code>	Dosyaya ekler (sona)
<code>dosya_var_mi("yol")</code>	<code>dogru</code> / <code>yanlis</code> döner
<code>klasor_olustur("yol")</code>	Klasör oluşturur
<code>zip_olustur "arsiv.zip"</code>	Boş ZIP oluşturur
<code>zip_ekle "arsiv.zip" "dosya"</code>	ZIP'e dosya ekler
<code>zip_listele "arsiv.zip" degisken</code>	İçeriği listeler
<code>zip_cikar "arsiv.zip" "klasor"</code>	ZIP'i açar
<code>zip_sil "arsiv.zip"</code>	ZIP'i siler

Sonraki bölüm: `03_AG_ISLEMLERI.md` — HTTP istekleri, JSON, dosya indirme ve ağ özellikleri.